



Sir Syed University of Engineering & Technology, Karachi

Mobile Application Development

Lecture 3: Mobile Application Development – Props, State and Hooks



Objective

- Props
- Prop Call backs
- State
- Hooks



Props (Properties)

- Inputs to a React component that allow you to customize how it looks or behaves.
- It's a way to pass data from a parent component to a child component
- They are read-only i.e. child component cannot change them.
- Props Offers
 - Reusability → You can reuse one component with different data.
 - Flexibility → Components adapt to different inputs.
 - Separation → Parent handles logic; child handles display

Props (Properties) - Example

```
import React from 'react';
import { View, Text } from 'react-native';

function Greeting(props) {
  return (
    <View>
      <Text>Hello, {props.name}!</Text>
    </View>
  );
}

export default function App() {
  return (
    <View>
      <Greeting name="Ahmed" />
      <Greeting name="Ali" />
    </View>
  );
}
```

Props (Properties) - Example



Component	Description
App	Parent component
Greeting	Child component
name	Prop being passed from parent to child
props.name	Value accessed inside the child

Output

Hello, Ahmad!
Hello, Ali!



Props (Properties)

- Props can be of multiple data types
 - Strings (name="Ali")
 - Numbers (age={20})
 - Booleans (isOnline={true})
 - Arrays (hobbies=[{"Music", "Coding"}])
 - Objects (user={{ name: "Ali", age: 20 }})
 - Functions (onPress={() => alert("Clicked!")})

Props (Properties) – Example 1

```
function UserCard({ name, age, country }) {  
  return (  
    <View>  
      <Text>{name}</Text>  
      <Text>Age: {age}</Text>  
      <Text>Country: {country}</Text>  
    </View>  
  );  
}  
  
export default function App() {  
  return <UserCard name="Ali" age={21} country="Pakistan" />;  
}
```



Props (Properties) – Example 1

Output:

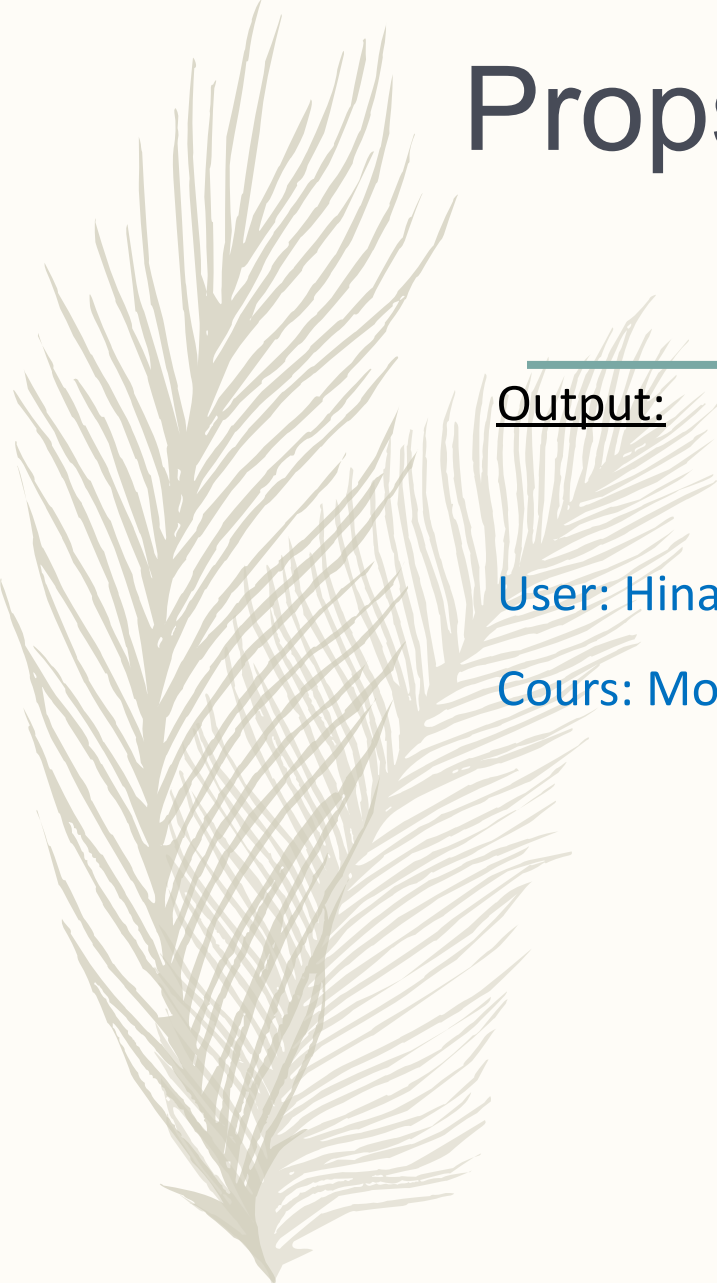
Ali

Age: 21

Country: Pakistan

Props (Properties) – Example 2

```
function InfoDisplay({ user, course }) {  
  return (  
    <View>  
      <Text>User: {user.name}</Text>  
      <Text>Course: {course.title}</Text>  
    </View>  
  );  
}  
  
export default function App() {  
  const user = { name: 'Hina', email: 'hina@mail.com' };  
  const course = { title: 'Mobile Application Development', duration: '8 weeks' };  
  return <InfoDisplay user={user} course={course} />;  
}
```



Props (Properties) – Example 2

Output:

User: Hina

Cours: Mobile Application Development



Callback Props

- When a parent component passes a function (as a prop) to the child component, so that the child can send data back or trigger an action in the parent.

Prop Callbacks - Example

```
// Parent Component
function Parent() {
  const handleChildClick = (message) => {
    alert("Message from child: " + message);
  };
  return <Child onClick={handleChildClick} />;
}

// Child Component
function Child(props) {
  return (
    <Button
      title="Click Me"
      onPress={() => props.onClick("Hello Parent!")}
    />
  );
}
```



State

- **Built-in object** that stores **data that can change over time** inside a component.
- Whenever the **state changes**, the component **automatically re-renders** to reflect those updates on the screen.
- State Is Local (Private) to a Component. Each component manages its own state.
- State Examples are battery level, volume, or WiFi status (changes while using)
- State Usages
 - Makes UI dynamic and interactive
 - Enables real-time updates (without reloading)
 - React automatically updates the screen when the state changes
 - Foundation for features like forms, animations, toggles, counters, and API data



State

```
import React, { useState } from 'react';
import { View, Text, Button } from 'react-native';

export default function Counter() {
  const [count, setCount] = useState(0); // state variable

  return (
    <View>
      <Text>Count: {count}</Text>
      <Button title="Increase" onPress={() => setCount(count + 1)} />
    </View>
  );
}
```



State Usages

Example	Description
count	Number that increases when a button is pressed
isLoading	Show spinner while data loads
text	Store input field value
isDarkMode	Switch between light and dark themes
user	Store logged-in user data



Props v/s State

Feature	Props	State
Who owns it?	Parent component	The component itself
Can it change?	No (read-only)	Yes (mutable)
Updates cause re-render?	Yes	Yes
Usage	Pass data into component	Manage data inside component
Analogy	Function parameter	Local variable inside function



React Hooks

- Hooks are **built-in functions** that let you use React features — such as **state**, **side effects**, or **context** — without writing a class
- Hooks allow **functional components** to:
 - Store and update data (**useState**)
 - Run code at specific points (**useEffect**)
 - Share logic between components (**custom hooks**)



Commonly used React Hooks

Hook	Purpose
<code>useState()</code>	Manage local state inside a component
<code>useEffect()</code>	Handle side effects (e.g., API calls, timers)
<code>useContext()</code>	Share data between components without props
<code>useRef()</code>	Access or store DOM elements and mutable values
<code>useMemo()</code>	Optimize performance by memoizing results
<code>useCallback()</code>	Optimize performance for functions
<code>useReducer()</code>	Advanced state management (like Redux light)



Arrow Functions

- An arrow function is a shorter and simpler way to write a function in JavaScript. It was introduced in ES6 (ECMAScript 2015).

- Example:

Conventional Style

```
function sayHello() {  
  console.log("Hello!");  
}
```

- Arrow Function

```
const sayHello = () => {  
  console.log("Hello!");  
};
```



Arrow Functions

- Example:

- Conventional Style

```
function add(a, b) {  
  return a + b;  
}
```

- Arrow Function

```
const add = (a, b) => {  
  return a + b;  
};    OR
```

```
const add = (a, b) => { return a + b; };
```

UseEffect Hook

- Runs side effects like fetching data, timers, or updating the DOM.

- Syntax

```
useEffect(() => {
```

```
  // 👉 - side effect code (runs when dependencies change)
```

```
}, [/👉 dependency array goes here */]);
```

- The first argument to `useEffect` is a function (an effect callback).
- This callback contains the code for the side effect you want to run. React will call this function at specific times (see the dependency array later).



UseEffect Hook

-
- Second argument is dependency array. It controls when the effect runs:
 - [] (empty): run the effect only once after the first render (mount). Cleanup runs on unmount.
 - [a, b]: run the effect after the first render and whenever a or b change. Cleanup runs before each re-run and on unmount.
 - Example: Run once when component is loaded

```
useEffect(() => {  
  console.log("Component mounted!");  
  return () => console.log("Component unmounted!");  
}, []);
```



React Hooks Examples

```
import React, { useState, useEffect } from 'react';
import { Text, View } from 'react-native';

export default function Timer() {
  const [seconds, setSeconds] = useState(0);
  useEffect(() => {
    const timer = setInterval(() => setSeconds(s => s + 1), 1000);
    return () => clearInterval(timer); // cleanup
  }, []);
  return (
    <View>
      <Text>Time: {seconds}s</Text>
    </View>
  );
}
```



React Hooks Examples

Output:

Time: 0s

Time: 1s

Time: 2s

Time: 3s

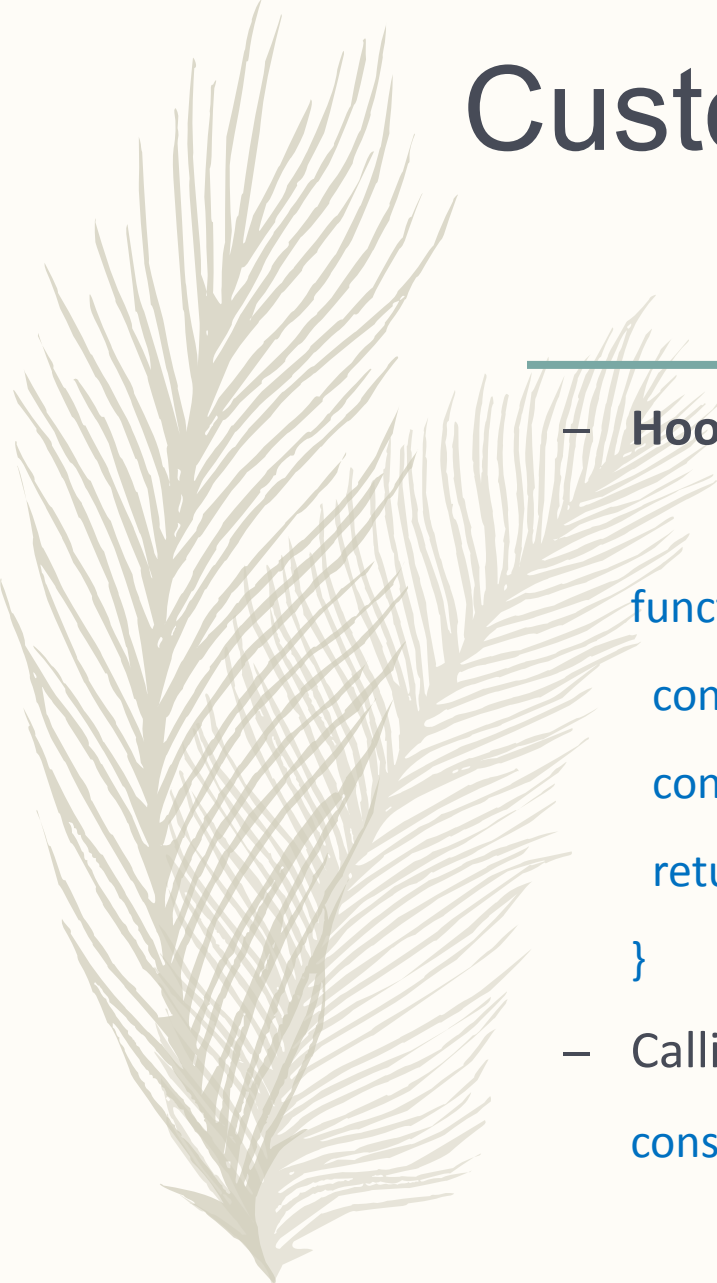
...

React Hooks Examples

useState → keeps track of seconds

useEffect → runs a timer (side effect)





Custom Hooks

- **Hooks** can be created to reuse logic between components

```
function useCounter(initialValue = 0) {  
  const [count, setCount] = useState(initialValue);  
  const increment = () => setCount(count + 1);  
  return { count, increment };  
}
```

- Calling

```
const { count, increment } = useCounter(5);
```